PortfolioGenie

Requirements Specification

Functional and Non-Functional Requirements

Digital Egypt Pioneers Initiative (DEPI) - Graduation Project
Phase: Requirements Gathering
Version 1.0 - October 2026

1. Introduction

1.1 Purpose

This document defines the functional and non-functional requirements of PortfolioGenie. It is the reference for system analysis, design, implementation, and testing, and it is aligned with the User Stories and Use Cases prepared by the team.

1.2 Project Overview

PortfolioGenie is a web platform that helps junior developers build a professional, high-performance portfolio. It analyzes the developer's GitHub activity and uses AI (prompt engineering) to generate tailored content: improved project descriptions (SEO-friendly and readable) and a personal "About Me" section based on the developer's coding style.

1.3 Scope

In scope:

- User registration, authentication and profile management.
- GitHub data import and analysis.
- Multi-step portfolio editor with a live preview.
- AI-assisted content generation and improvement.
- Portfolio publishing and export.

Out of scope: payments, custom domain hosting, and support for platforms other than GitHub (e.g. GitLab, Bitbucket).

1.4 Definitions and Abbreviations

Term	Definition
FR	Functional Requirement
NFR	Non-Functional Requirement
JWT	JSON Web Token, used for stateless authentication
REST API	Web service interface used for communication between frontend and backend
SEO	Search Engine Optimization
WCAG	Web Content Accessibility Guidelines
AI Prompt	Structured instruction sent to the AI model to generate or improve content

2. Overall Description

2.1 Target Users

Actor	Description
Junior Developer (primary)	Registers, connects GitHub, builds, previews, publishes, and exports a portfolio.
Visitor	Views a published portfolio through its public link (no account needed).
GitHub API (external)	Provides repositories, languages and activity data.
AI Service (external)	Generates and improves text content from prompts.

2.2 Technology Constraints (from the program checklist)

- Frontend: React with TypeScript, React Hooks (including Custom Hooks) and React Router.
- Backend: Node.js and Express exposing RESTful APIs.
- Deployment: Docker Compose managing the frontend, backend and database services.
- Documentation: Functional documentation including User Stories and Acceptance Criteria.

2.3 Assumptions and Dependencies

- The user has a public GitHub account with at least one repository.
- The system depends on the availability of the GitHub API and the AI service.
- Internet access is required to use the platform.
- GitHub API rate limits apply and must be handled gracefully.

3. Functional Requirements

Priority levels: High = essential for the first release, Medium = important, Low = nice to have.

3.1 Authentication and Account Management

ID	Requirement	Priority
FR-01	The system shall allow a user to register with name, email and password.	High
FR-02	The system shall allow a registered user to log in and receive a secure JWT session.	High
FR-03	The system shall allow a user to log out, ending the session.	High
FR-04	The system shall allow a user to log in using their GitHub account.	Medium
FR-05	The system shall allow a user to view and update their profile information.	Medium
FR-06	The system shall allow a user to reset a forgotten password.	Low

3.2 GitHub Integration and Analysis

ID	Requirement	Priority
FR-07	The system shall accept a GitHub username (or connected account) and fetch the user's public repositories.	High
FR-08	The system shall analyze repository data: languages used, stars, topics, last activity and commit frequency.	High
FR-09	The system shall display the analysis results as a summary (e.g. top languages and key projects).	Medium
FR-10	The system shall allow the user to select which repositories to include in the portfolio.	High
FR-11	The system shall show a clear message when GitHub data cannot be retrieved (invalid username, rate limit or network error).	High

3.3 Portfolio Editor (Multi-Step Form)

ID	Requirement	Priority
FR-12	The system shall provide a multi-step form covering: Personal Info, Skills, Projects, Experience and Education, and Contact and Social Links.	High
FR-13	The system shall validate the inputs of each step before allowing the user to continue.	High

ID	Requirement	Priority
FR-14	The system shall allow the user to move forward and backward between steps without losing entered data.	High
FR-15	The system shall pre-fill form fields (projects, skills) using the imported GitHub data.	Medium
FR-16	The system shall allow the user to save a draft and resume editing later.	Medium
FR-17	The system shall show a progress indicator for the current step.	Low

3.4 AI Content Generation

ID	Requirement	Priority
FR-18	The system shall improve a project description to be clearer, readable and SEO-friendly using AI.	High
FR-19	The system shall suggest a professional "About Me" section based on the developer's coding style and GitHub activity.	High
FR-20	The system shall let the user accept, edit or reject each AI suggestion.	High
FR-21	The system shall allow the user to regenerate a suggestion.	Medium
FR-22	The system shall show a loading state while the AI response is pending and a clear message if it fails.	High

3.5 Live Preview and Templates

ID	Requirement	Priority
FR-23	The system shall provide a live preview that reflects editor changes without a page reload.	High
FR-24	The system shall use client-side routing to navigate between the editor and the preview.	High
FR-25	The system shall let the user choose from available portfolio templates or themes.	Medium
FR-26	The system shall allow previewing the portfolio in desktop and mobile layouts.	Low

3.6 Publishing and Export

ID	Requirement	Priority
FR-27	The system shall publish the portfolio to a unique public URL.	High
FR-28	The system shall allow the user to unpublish or delete their portfolio.	Medium
FR-29	The system shall allow the user to export the portfolio as static HTML/CSS files.	Medium
FR-30	The system shall allow a visitor to view a published portfolio without logging in.	High

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Pages shall load in under 3 seconds on a standard broadband connection.
NFR-02	Performance	The live preview shall update within 300 ms of an editor change.
NFR-03	Performance	Standard API responses (excluding AI and GitHub calls) shall return within 500 ms for 95% of requests.
NFR-04	Performance	AI-generated content shall be returned within 10 seconds, with a loading indicator shown meanwhile.
NFR-05	Performance	Published portfolios shall be lightweight and achieve a Lighthouse performance score of 90 or above.
NFR-06	Security	Passwords shall be stored hashed (e.g. bcrypt) and never in plain text.
NFR-07	Security	Authentication shall use JWT with an expiry time, and protected routes shall reject unauthenticated requests.
NFR-08	Security	All communication shall use HTTPS in production.
NFR-09	Security	All user inputs shall be validated and sanitized on the server to prevent XSS and injection attacks.
NFR-10	Security	API keys and secrets (AI service, GitHub) shall be kept in environment variables and never exposed to the client.
NFR-11	Security	The API shall apply rate limiting to protect authentication and AI endpoints from abuse.
NFR-12	Usability	The interface shall follow WCAG 2.1 Level AA (contrast, labels, alt text, focus states).
NFR-13	Usability	All main functions shall be usable with a keyboard only.
NFR-14	Usability	The interface shall be responsive on mobile, tablet and desktop screens.
NFR-15	Usability	Error and validation messages shall be clear and tell the user

ID	Category	Requirement
		how to fix the problem.
NFR-16	Usability	A new user shall be able to create a first portfolio draft in under 15 minutes without external help.
NFR-17	Reliability	The system shall target 99% availability during normal operation.
NFR-18	Reliability	The system shall degrade gracefully when GitHub or the AI service is unavailable (manual input remains possible).
NFR-19	Reliability	User drafts shall be saved so that data is not lost on refresh or connection loss.
NFR-20	Reliability	Containers shall restart automatically after a failure.
NFR-21	Maintainability	The code shall be written in TypeScript following clean code practices with ESLint and Prettier.
NFR-22	Maintainability	The code shall be modular (reusable components and Custom Hooks, layered backend).
NFR-23	Maintainability	Core logic shall be covered by automated tests.
NFR-24	Portability	The whole system shall run with a single Docker Compose command (frontend, backend, database).
NFR-25	Compatibility	The platform shall work on the latest versions of Chrome, Edge, Firefox and Safari.

5. Requirements Traceability

This section links requirements to the User Stories and Use Cases prepared by the team, so every requirement is covered and every story maps back to a requirement. The IDs below should be completed once the User Stories and Use Cases are finalized.

5.1 Proposed Use Case Groups

Group	Proposed Use Cases	Related FRs
Account	Register, Log in, Log out, Manage profile	FR-01 to FR-06
GitHub	Import repositories, View analysis, Select repositories	FR-07 to FR-11
Editor	Fill multi-step form, Save draft, Resume draft	FR-12 to FR-17
AI	Improve description, Generate About Me, Accept or reject suggestion	FR-18 to FR-22
Preview	View live preview, Switch template, Preview on mobile	FR-23 to FR-26

Group	Proposed Use Cases	Related FRs
Publish	Publish, Unpublish, Export, View public portfolio	FR-27 to FR-30

5.2 Traceability Matrix

Requirement	User Story ID	Use Case ID	Status
FR-01 to FR-06	TBD	TBD	Pending
FR-07 to FR-11	TBD	TBD	Pending
FR-12 to FR-17	TBD	TBD	Pending
FR-18 to FR-22	TBD	TBD	Pending
FR-23 to FR-26	TBD	TBD	Pending
FR-27 to FR-30	TBD	TBD	Pending

Key Performance Indicators (KPIs)

1. Purpose

These KPIs measure whether portfolioGenie is delivered on time, with the expected quality, and with a good user experience. Each KPI has a clear formula, a target, a data source and a review frequency so progress can be tracked objectively in the weekly meeting.

2. Success Criteria

- All six DEPI deliverables submitted on or before their deadlines (16 Oct, 6 Nov, 30 Nov, 4 Dec 2026).
- All must-have requirements implemented, with no critical bug open at final delivery.
- AI-generated portfolios are produced quickly and rated as useful by real users.
- The website loads fast and works correctly on the main browsers and on mobile screens.

3. KPI Table

1. Schedule

ID	KPI	Definition / Formula	Target	Measurement source	Frequency
K1	On-time milestone delivery	Milestones delivered by DEPI deadline / total milestones	**100% (6 of 6)**	Milestones sheet status	Per milestone
K2	Schedule variance	Planned % complete minus actual % complete (Gantt)	**Within +/- 10%**	Gantt % Complete column	Weekly
K3	Task completion rate	Tasks completed / tasks planned for the week	**>= 90%**	Task board	Weekly

2. Product Quality

ID	KPI	Definition / Formula	Target	Measurement source	Frequency
K4	Requirements coverage	Must-have requirements implemented / total must-have	**100%**	SRS traceability table	End of build
K5	Test pass rate	Passed test cases / executed test cases	**>= 90%**	Test report	Weekly in testing
K6	Critical bugs open at delivery	Number of open critical/blocker bugs	**0**	Bug tracker	Weekly
K7	Code review coverage	Merged pull requests reviewed / total pull requests	**>= 90%**	GitHub	Weekly

3. AI & Performance

ID	KPI	Definition / Formula	Target	Measurement source	Frequency
K8	Portfolio generation time	Average time from submit to generated portfolio	**<= 30 seconds**	Timed test runs (20 samples)	Per build
K9	Generated content quality	Average rating of AI output by 10 testers (1-5)	**>= 4.0 / 5**	Rating form	Before final demo
K10	Generation success rate	Successful generations / total attempts	**>= 95%**	App logs	Weekly in testing

4. User Satisfaction

ID	KPI	Definition / Formula	Target	Measurement source	Frequency
K11	Usability score (SUS)	System Usability Scale score from test users	**>= 70**	SUS questionnaire (min. 10 users)	UAT
K12	Task success rate in UAT	Users who build a portfolio without help / total users	**>= 80%**	UAT sessions	UAT

5. Website Performance & Compatibility

ID	KPI	Definition / Formula	Target	Measurement source	Frequency
K16	Page load time	Average load time of main pages on normal internet connection	**<= 3 seconds**	Chrome DevTools / Lighthouse	Per build
K17	Cross-browser compatibility	Browsers where all main flows work / browsers tested (Chrome, Edge, Safari)	**100% (3 of 3)**	Browser test checklist	Testing phase
K18	Mobile responsiveness	Main pages that display correctly on phone, tablet and desktop / total main pages	**100%**	Device / DevTools responsive tests	Testing phase

6. Team & Process

ID	KPI	Definition / Formula	Target	Measurement source	Frequency
K13	Risk exposure trend	Number of High/Critical risks open	**Decreasing; 0 Critical at delivery**	Risk register	Weekly
K14	Meeting attendance & participation	Meetings attended / meetings held	**>= 90%**	Meeting log	Weekly
K15	Documentation completeness	Required documents delivered / required documents	**100%**	DEPI deliverables checklist	Per phase

4. Tracking & Reporting

- The PM updates the KPI values every week and records them next to the Gantt chart progress.
- A KPI that misses its target for two consecutive weeks is discussed in the team meeting with a corrective action.
- Final KPI results are included in the final report and presentation (4 Dec 2026).

portfolioGenie - Project Gantt Chart

ID	Task	Owner	Start	End	Days	DEPI Deadline	Slack (days)	% Complete	28 Sep	5 Oct	12 Oct	19 Oct	26 Oct	2 Nov	9 Nov	16 Nov	23 Nov	30 Nov
1	**Project Planning & Management**		**1 Oct 2026**	**16 Oct 2026**	**16**	**16 Oct 2026**	**0**	**100%**										
1.1	Define scope, objectives & team roles	Rashed	1 Oct 2026	5 Oct 2026	5			100%										
1.2	Build Gantt chart, milestones & resource plan	Hazem	7 Oct 2026	16 Oct 2026	10			100%										
1.3	Risk assessment & KPIs definition	Hazem	7 Oct 2026	16 Oct 2026	10			100%										
1.4	Project Proposal (overview, problem, goals, scope)	Rashed	7 Oct 2026	16 Oct 2026	10			100%										
2	**Literature Review**		**7 Oct 2026**	**16 Oct 2026**	**10**	**16 Oct 2026**	**0**	**100%**										
2.1	Find 3-5 similar tools & collect trusted sources	Hind	7 Oct 2026	16 Oct 2026	10			100%										
2.2	Analyze pros/cons & identify gaps/opportunities	Hind	7 Oct 2026	16 Oct 2026	10			100%										
2.3	Final comparison & complete Literature_Review.docx	Walaa	7 Oct 2026	16 Oct 2026	10			100%										
3	**Requirements Gathering**		**7 Oct 2026**	**16 Oct 2026**	**10**	**16 Oct 2026**	**0**	**100%**										
3.1	Identify stakeholders & their needs	Walaa	7 Oct 2026	16 Oct 2026	10			100%										
3.2	Functional & non-functional requirements (Requirements.docx)	Mostafa	7 Oct 2026	16 Oct 2026	10			100%										
3.3	User stories, use cases & consistency review	Walaa + Mostafa	7 Oct 2026	16 Oct 2026	10			100%										
4	**System Analysis & Design**		**19 Oct 2026**	**6 Nov 2026**	**19**	**6 Nov 2026**	**0**	**0%**										
4.1	Use case, ERD & DFD diagrams	Team (TBD)	19 Oct 2026	28 Oct 2026	10			0%										
4.2	System architecture & AI pipeline design	Team (TBD)	21 Oct 2026	30 Oct 2026	10			0%										
4.3	UI/UX wireframes & prototype (frontend design first)	Team (TBD)	19 Oct 2026	3 Nov 2026	16			0%										
4.4	Database schema & API design	Team (TBD)	28 Oct 2026	6 Nov 2026	10			0%										
5	**Implementation (Source Code & Execution)**		**9 Nov 2026**	**30 Nov 2026**	**22**	**30 Nov 2026**	**0**	**0%**										
5.1	1st: Frontend development (on mock data)	Team (TBD)	9 Nov 2026	16 Nov 2026	8			0%										
5.2	2nd: Backend & database development	Team (TBD)	17 Nov 2026	23 Nov 2026	7			0%										
5.3	3rd: AI portfolio-generation module	Team (TBD)	24 Nov 2026	27 Nov 2026	4			0%										
5.4	Integration & deployment	All	28 Nov 2026	30 Nov 2026	3			0%										
6	**Final Presentation & Testing & Reports**		**23 Nov 2026**	**4 Dec 2026**	**12**	**4 Dec 2026**	**0**	**0%**										
6.1	Testing (unit, integration, UAT)	Team (TBD)	23 Nov 2026	1 Dec 2026	9			0%										
6.2	Responsive & cross-browser testing (mobile, Chrome, Edge, Safari)	Team (TBD)	27 Nov 2026	2 Dec 2026	6			0%										
6.3	Final report & documentation	Team (TBD)	25 Nov 2026	3 Dec 2026	9			0%										
6.4	Presentation preparation & rehearsal	All	1 Dec 2026	4 Dec 2026	4			0%										

Milestones & Deliverables

ID	Milestone	Deliverables	DEPI Deadline	Owner	Done?	Days Left	Status
M1	Project plan approved	Project_Plan.xlsx (Gantt), Risk_Assessment.docx, KPIs.docx, Project_Proposal.docx	16 Oct 2026	Hazem + Rashed	Yes	1	Completed
M2	Literature review submitted	Literature_Review.docx: 3-5 similar tools, sources, pros/cons, gaps & opportunities	16 Oct 2026	Hind + Walaa	Yes	1	Completed
M3	Requirements baseline	Requirements.docx: stakeholders, FR & NFR, user stories, use cases	16 Oct 2026	Walaa + Mostafa	Yes	1	Completed
M4	Analysis & design complete	UML (use case, ERD, DFD), architecture, wireframes, API spec	6 Nov 2026	BA + Backend + Frontend	No	22	On track
M5	Implementation complete	Source code, working deployed prototype, execution guide	30 Nov 2026	All dev roles	No	46	On track
M6	Final delivery	Test report, final project report, final presentation	4 Dec 2026	All	No	50	On track

Resource Plan

Role	Team Member	Responsibilities	P1 Plan	P2 Lit	P3 Req	P4 Design	P5 Impl	P6 Test/Rpt	Impl. Order	Impl. Start	Impl. End	Tools
PM	Hazem	Planning, Gantt, risk & KPI follow-up, schedule tracking	X	X	X			X	-			Excel, Trello/Jira, GitHub
Team Leader / Proposal	Rashed	Team lead, scope & team roles, Project Proposal: overview, problem, goals, users	X						-			Word, Google Docs
Research	Hind	Similar tools research, sources, pros/cons, gaps		X					-			Google Scholar, Zotero
BA	Walaa	Final comparison, stakeholders, user stories & use cases		X	X				-			Draw.io, Google Docs
BA / Requirements	Mostafa	Functional & non-functional requirements, Requirements.docx			X				-			Word, Draw.io
Frontend	TBD	UI/UX design, portfolio templates, responsive web app (built first, on mock data)				X	X		1st	9 Nov 2026	16 Nov 2026	Figma, React, Tailwind
Backend	TBD	DB, APIs, authentication, deployment (starts after Frontend)				X	X		2nd	17 Nov 2026	23 Nov 2026	Node.js/Python, PostgreSQL, Docker
AI	TBD	LLM integration, content generation quality (starts after Backend)				X	X		3rd	24 Nov 2026	27 Nov 2026	LLM API, Python
QA	TBD	Test plan, test cases, responsive & cross-browser testing, UAT						X	-			Postman, Selenium/Jest, Lighthouse

Risk Assessment & Mitigation Plan

1. Purpose

This document identifies the main risks that could affect the delivery of portfolioGenie (an AI-powered portfolio generator), scores them, and defines how the team will prevent, reduce or respond to each one. It supports the Gantt chart in Project_Plan.xlsx and the DEPI deadlines (Planning 16 Oct, Design 6 Nov, Implementation 30 Nov, Final delivery 4 Dec 2026).

2. Scoring Method

Each risk is rated for Probability (1 = very unlikely, 5 = almost certain) and Impact (1 = negligible, 5 = project failure). Risk Score = Probability x Impact.

Score	Level	Required action
15 - 25	**Critical**	Act immediately; PM reviews weekly
10 - 14	**High**	Mitigation plan active; reviewed weekly
5 - 9	**Medium**	Monitor; reviewed every two weeks
1 - 4	**Low**	Accept; review at phase end

3. Risk Summary

Total risks identified: 14 | Critical: 1 | High: 6 | Medium: 7 | Low: 0

4. Risk Register

ID	Category	Risk description	Prob.	Impact	Score	Level	Mitigation (prevent / reduce)	Contingency (if it happens)	Owner
R1	Schedule	Tight DEPI deadlines; tasks overrun (esp. Implementation by 30 Nov, with Frontend, Backend and AI built one after another)	4	4	16	**Critical**	Weekly progress check against Gantt; MoSCoW prioritization; 1-week buffer before each DEPI deadline	Cut Could-have features; add parallel work	Rashed / Hazem
R2	Scope	Scope creep - extra portfolio features added mid-project	4	3	12	**High**	Freeze requirements after SRS (16 Oct); change-request log approved by PM	Move extras to a 'future work' list	Rashed / Walaa
R3	Technical	LLM output quality is inconsistent or inaccurate for generated portfolio content	3	4	12	**High**	Structured prompts + templates; human edit step; test with sample profiles; start prompt prototyping during Design phase because the AI build window is short (24-27 Nov)	Fallback to template-based generation with user-filled fields	AI
R4	Technical	AI API cost, rate limits or downtime	3	4	12	**High**	Use free/low-tier quota; cache results; retry & timeout handling	Switch provider or use a smaller/open model	AI / Backend (TBD)
R5	Technical	Integration problems: frontend is built first on mock data, so it may not match the real backend/AI responses	3	3	9	**Medium**	Agree API contract (JSON format) before frontend starts; mock data follows it; weekly integration builds once backend starts	Dedicated integration window (28-30 Nov); keep mock-data version as fallback for the demo	Backend
R6	Resource	Team member unavailable (illness, exams, other commitments)	3	4	12	**High**	Cross-train on key modules; document work; shared repo	Reassign tasks; reduce scope of non-core features	Rashed
R7	Resource	Skill gaps with chosen technologies / AI tools	3	3	9	**Medium**	Early spikes during Literature Review; tutorials; mentor support	Choose simpler stack	All
R8	Requirements	Requirements unclear or change after user feedback	3	3	9	**Medium**	Survey real users; validate SRS with mentor; prototype early	Re-prioritize backlog in change meeting	Walaa / Mostafa
R9	Quality	Insufficient testing time before 4 Dec	3	4	12	**High**	Start test cases in design phase; test continuously during implementation	Focus testing on critical user flows	QA
R10	Security / Legal	Leakage of user personal data; AI-generated text copyright concerns	2	5	10	**High**	Minimal data collection; HTTPS; env secrets; privacy notice; check content licences	Delete data on request; remove disputed content	Backend
R11	Infrastructure	Hosting/deployment failure or data loss before demo	2	4	8	**Medium**	Git + regular backups; deploy early (staging); documented deployment steps	Run demo locally / recorded demo as backup	Backend
R12	Communication	Miscommunication or delayed decisions inside team and with DEPI mentor	3	2	6	**Medium**	Weekly meeting, shared task board, single source of truth for documents	Escalate to mentor; daily sync in critical weeks	Rashed
R13	Technical	Website looks or works differently across browsers (Chrome, Edge, Safari) and on mobile screens	3	3	9	**Medium**	Responsive design from the start (mobile-first); test on 3 browsers and 2 screen sizes each sprint	Fix critical layout bugs first; drop non-essential animations/features	Frontend (TBD)

R14	Performance	Website loads slowly or AI generation makes pages hang (heavy images, long API calls)	3	3	9	**Medium**	Optimize images, lazy loading, loading indicators, API timeouts; measure load time in testing	Cache generated results; simplify templates	Frontend / Backend (TBD)

5. Monitoring & Review

- The PM reviews the risk register in the weekly team meeting and updates probability, impact and status.
- Any risk that reaches Critical triggers an immediate meeting and is reported to the DEPI mentor.
- New risks are added with an ID, owner and mitigation; closed risks are kept in the log with the closing date.
- Risk review checkpoints: end of each phase (16 Oct, 6 Nov, 30 Nov, 4 Dec 2026).

6. Resource & Schedule Coordination

Owners for planning, research and requirements risks are named team members (Hazem, Hind, Walaa, Mostafa, Rashed); owners shown as Backend, AI, Frontend or QA are roles to be assigned once the implementation split is agreed (see the Resource Plan sheet in Project_Plan.xlsx). A one-week buffer is kept before each DEPI deadline, and the last week of November is reserved for integration.